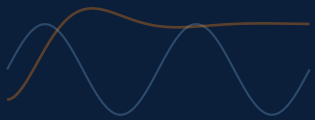
WEEK 2 · DAY 3 / 5

Mechatronics and Robotics Engineering (50 h)



Robotic Manufacturing and Flexible Automation

5.2 Robot applications ▪ 5.3 Work-cell design ▪ 5.4 PLCs & ladder logic ▪ 5.5 Robot programming

Mechatronics and Robotics Engineering Programme

Week 2 · Day 3 ▪ 2 weeks · 10 days · 50 hours

Agenda and learning outcomes

Code	Session	Net time
5.2	Industrial robot applications	1.0 h
5.3	Work-cell design: layout, reach, cycle time, tooling	1.0 h
5.4	PLCs: scan cycle, I/O, ladder logic, sequences	1.5 h
5.5	Robot programming: pendant, frames, offline, simulation	1.5 h
	Total (+ 10-min breaks about every hour)	**5.0 h**

Running example all day: a small cell where a 6-axis robot takes machined parts from a CNC lathe, checks them and stacks them on a pallet – coordinated by a PLC.

By the end of today you can . . .

- match robot types to typical applications
- sketch a work-cell layout and check reach
- calculate cycle time, takt time and grip force
- explain the PLC scan cycle and read simple ladder logic
- write a start/stop latch and a small sequence
- explain base, tool and user frames and teach a pick-and-place program
- compare online and offline programming

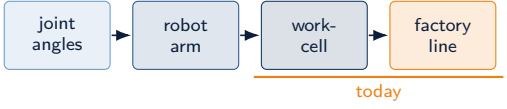
What we learned so far: from one arm to the factory

Yesterday – the robot arm (4.4–4.7)

- **Inverse kinematics:** from a desired gripper pose to joint angles (often two answers: elbow up / down).
- **Jacobian:** joint speeds → tool speed; at a **singularity** the arm loses a direction.
- **Dynamics:** payload and speed demand torque.
- **Moves:** PTP (joint space) vs. linear/circular (Cartesian space).

Yesterday – automation (5.1)

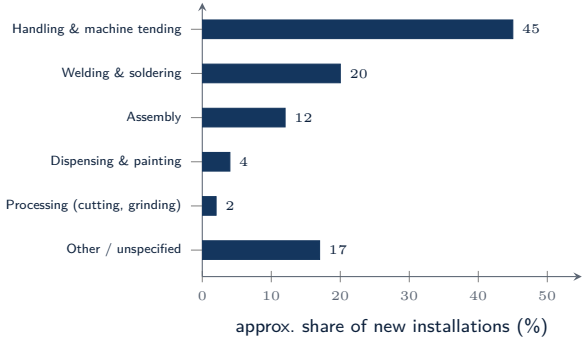
- **Fixed** automation: one product, very fast (transfer line).
- **Programmable:** batches, re-program between batches.
- **Flexible:** product mix, quick change-over (FMS).
- Industry 4.0: connected machines and data.



Session 1

5.2 Industrial Robot Applications

Where do industrial robots work?



Rounded, order-of-magnitude values; see IFR *World Robotics* for yearly data.

- Almost half of all robots simply **move things**: pick, place, load, stack.
- **Welding** grew up with the car industry and is still a big user.
- Robots take the jobs that are **dull, dirty, dangerous** – or need **precision and repeatability**.
- Main customers: automotive and electronics, then metal, plastics, food.

Take-away

Most applications are "move a part from A to B, accurately, all day".

What decides which robot fits a task?

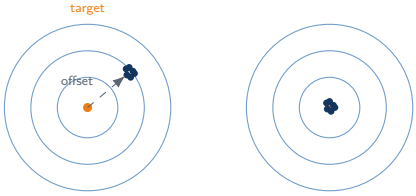
Application	Typical robot	What matters most	Typical numbers
Spot welding	6-axis, large	payload (heavy gun), stiffness	150–300 kg payload
Arc welding	6-axis, small	smooth path, constant speed	5–15 mm/s travel
Painting	6/7-axis, hollow wrist	smooth path, explosion protection	0.3–1 m/s gun speed
Palletising	4-axis or 6-axis, large	payload, reach, speed	50–700 kg, 3 m reach
Machine tending	6-axis or gantry	reach into the machine, reliability	10–50 kg
Assembly	SCARA, 6-axis, cobot	repeatability, compliance	$\pm$0.01–0.05 mm
Pick-and-place	delta, SCARA	speed	100+ picks/min

Data-sheet vocabulary

Payload: mass at the wrist incl. gripper. **Reach**: max. distance of the wrist centre. **Repeatability**: how well it returns to the same taught point.

Going further: Robot performance tests (repeatability, accuracy, path) are defined in ISO 9283.

Accuracy vs. repeatability: two different promises



repeatable, not accurate

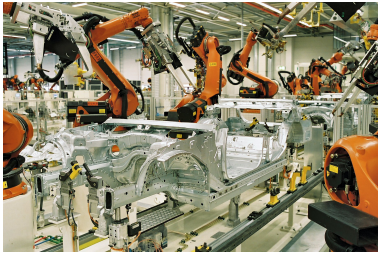
accurate and repeatable

- **Repeatability**: spread of many returns to the *same taught* point. Typical: ± 0.02–0.1 mm.
- **Accuracy**: error when going to a *calculated* point (from CAD). Often 0.5–1 mm without calibration.
- Teaching by hand only needs repeatability; offline programs need accuracy (session 5.5).

Take-away

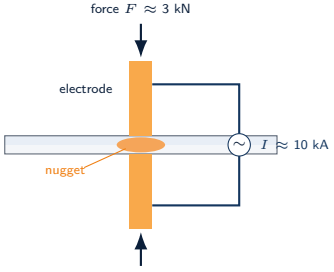
A robot is usually far more *repeatable* than *accurate*. Like a darts player who always hits the same spot – but not the bull's-eye.

Spot welding: the car body's 4000 dots



Spot welding of car bodies with KUKA robots. BMW Werk Leipzig, CC BY-SA 2.0 de, via Wikimedia Commons

- Two copper electrodes squeeze the sheets, a large current melts a small "nugget".
- The heavy **weld gun** (50–150 kg) sets the robot size.

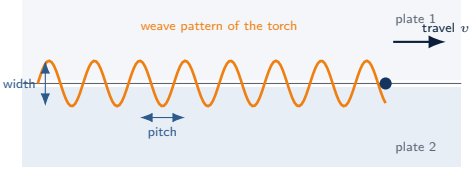


Heat in the joint: $Q = I^2\,R\,t$ $\quad (\mathrm{J} = \mathrm{A}^2 \cdot \Omega{\cdot}\mathrm{s})$

Example: $I = 10\,\mathrm{kA}$, contact resistance $R = 100\,\mu\Omega$, $t = 0.2\,\mathrm{s}$:

$Q = (10^4)^2 \cdot 10^{-4} \cdot 0.2 =$ **2000 J** – in a 6 mm spot!

Arc welding: a smooth path at constant speed



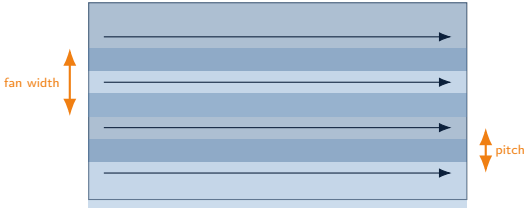
- The torch must follow the seam with a fixed angle and distance.
- Speed too slow $\rightarrow$ burn-through; too fast $\rightarrow$ weak seam.
- Robot uses **linear** and **circular** moves (4.7), often with **weaving**.

Worked example – seam time

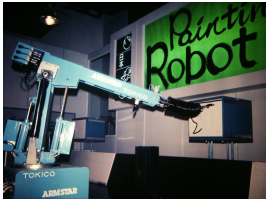
1. Seam length $L = 1.2\,\text{m} = 1200\,\text{mm}$.
2. Travel speed $v = 8\,\text{mm/s}$.
3. Arc-on time $t = L/v = 1200/8 = \mathbf{150\,s}$.
4. Add air moves and torch cleaning (say 30 s) $\rightarrow$ 180 s per part.

Going further: Seam tracking (touch sensing, laser line scanners, through-arc sensing) corrects the path when real parts differ from the CAD model.

Painting and coating: even layers, no sparks



- Constant gun speed, distance and **overlap** give even thickness.
- Paint mist is explosive: motors are **explosion-proof** (pressurised).
- Hollow wrist keeps hoses inside the arm.

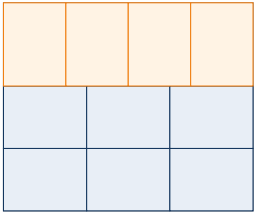


Painting robot (Japan, 1982 World's Fair). Gary Todd, CC0, via Wikimedia Commons

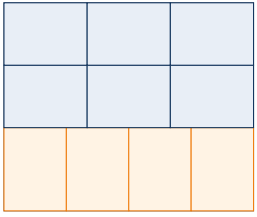
Worked example – door panel

1. Fan width 300 mm, 50 % overlap → pitch 150 mm.
2. Panel height 1.5 m: $1500/150 = 10$ passes.
3. Length 2 m at 0.5 m/s: 4 s per pass → **40 s** + turn-arounds.

Palletising: stacking boxes in stable patterns



layer A



layer B (mirrored)

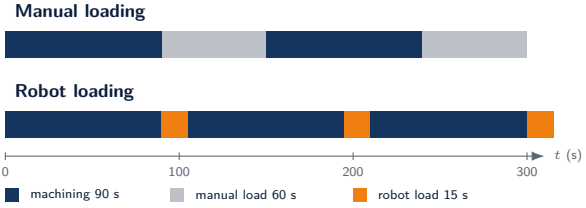


Robots palletising bread at a bakery. KUKA Roboter GmbH, Bachmann, public domain, via Wikimedia Commons

- Alternating layers **interlock** – like bricks in a wall.
- Robot program = one pick position + a pattern of place positions (5.5).
- Typical: 4-axis arm (wrist always vertical), vacuum or fork gripper.

> **Rate check:** 12 boxes per minute → one pick-and-place every 5 s.

Machine tending: keep the expensive machine busy



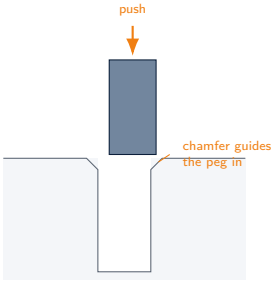
Machine utilisation $U = \dfrac{t_{\text{machining}}}{t_{\text{machining}} + t_{\text{load}}}$

- Robot opens the door, unloads the finished part, loads a raw part, closes the door, starts the cycle.
- Runs nights and weekends ("lights-out").

Worked example

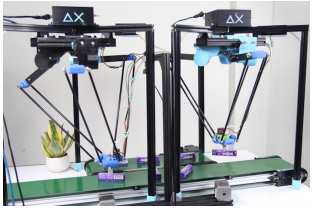
1. Manual: $U = 90/(90+60) = 60\,\%$.
2. Robot: $U = 90/(90+15) = \mathbf{86\,\%}$.
3. Same machine, $+43\,\%$ more parts.

Assembly and pick-and-place: speed and gentle precision



peg-in-hole: clearance $\approx$ 0.01–0.05 mm

- Assembly needs **compliance**: a little "give" so small misalignments don't jam.
- Chamfers, compliant wrists or force control (2.9) help.



Delta robots picking from a conveyor. Trungdoanhong, CC BY-SA 4.0, via Wikimedia Commons

- **Pick-and-place**: light parts, very fast – delta and SCARA robots.
- 120 picks/min $\rightarrow$ **0.5 s** per pick incl. up, across, down, release.

Going further: A remote-centre-compliance (RCC) wrist lets the peg rotate about its tip, so contact forces straighten it.

Check your understanding – 5.2

Check your understanding

1. Why is the robot for spot welding usually much bigger than the robot for arc welding on the same car?
2. A robot always returns within ± 0.03 mm of a taught point, but misses a CAD-calculated point by 0.8 mm. Which property is good, which is poor?
3. A CNC cycle is 60 s; manual loading takes 40 s, robot loading 10 s. Calculate both machine utilisations.

Discuss with your neighbour for 2 minutes.

Summary 5.2 – Industrial robot applications

Key take-aways

- About half of robots do handling; welding is next.
- The process sets the robot: payload, reach, path quality, speed, environment.
- Spot welding: heavy gun; arc welding and painting: smooth, constant-speed paths.
- Palletising and pick-and-place: patterns and speed; assembly: compliance.
- Robots are more repeatable than accurate.

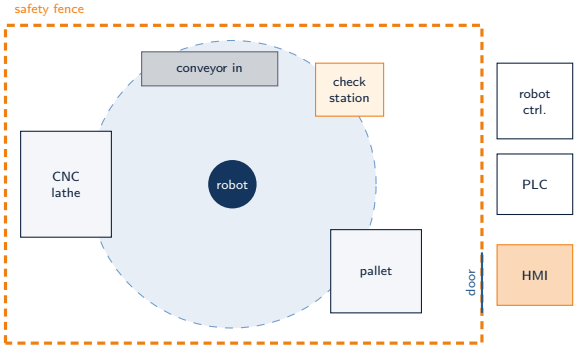
Key terms

Payload – mass the wrist may carry.
Reach – max. wrist distance.
Repeatability – return to a taught point.
Accuracy – hit a calculated point.
Machine tending – loading a machine.
Compliance – controlled "give".

Session 2

5.3 Robotic Work-Cell Design

What is a robotic work-cell?

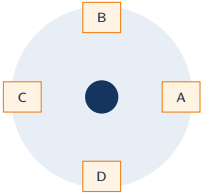


- **Work-cell**: robot + everything it needs to do one job on its own.
- Robot, **end-of-arm tooling** (EOAT), fixtures, feeders/conveyors, process equipment.
- Control: robot controller, **PLC**, operator panel (**HMI**).
- Guarding: fence, interlocked door, light curtain (W2D4).

Analogy

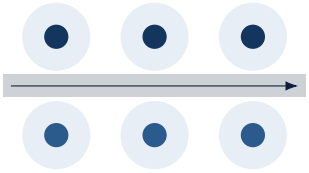
A kitchen: the cook (robot) is only fast if pots, knives and fridge are all within arm's reach.

Cell layouts: three common patterns



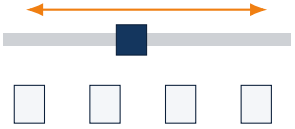
robot-centred

stations around one robot



in-line

robots along a conveyor (car body line)

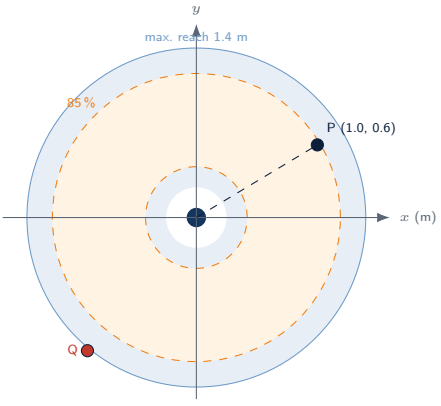


robot on a linear track

one robot serves several machines

- **Robot-centred:** compact, cheap, one robot = bottleneck.
- **In-line:** high volume; each robot does a small piece of work.
- **Track / gantry / AMR:** extends reach; adds an extra (7th) axis.
- Choose by volume, part size and floor space.

Reach check: is every station inside the comfort zone?



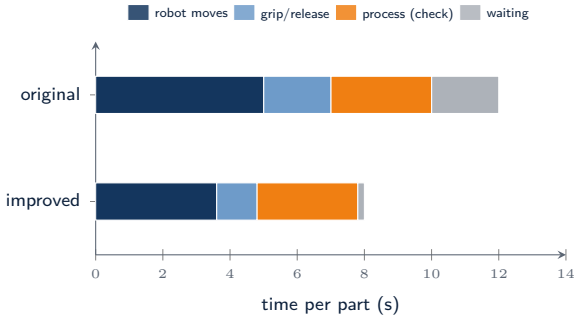
- Near full reach the arm is stretched (close to a **singularity**, slow, little stiffness).
- Too close to the base the arm folds onto itself.
- Rule of thumb: place stations at **30–85 %** of max. reach, at a good height.

Worked example

1. Point P: $r = \sqrt{1.0^2 + 0.6^2} = 1.17\,\text{m}$.
2. Ratio: $1.17/1.4 = 83\,\% \rightarrow$ OK, but at the limit.
3. Point Q $(-0.9, -1.1)$: $r = 1.42\,\text{m} > 1.4\,\text{m} \rightarrow$ **unreachable**; move the station.

Going further: Reach is not enough: the tool also needs the right *orientation* at that point. Check it in simulation (5.5).

Cycle time: where do the seconds go?



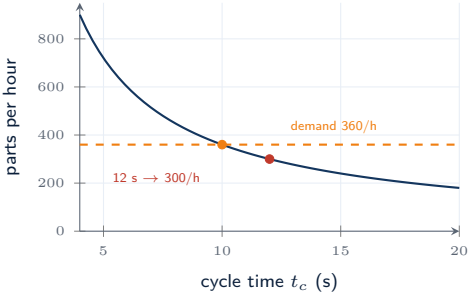
- **Cycle time**: time between two finished parts leaving the cell.
- Biggest savings usually come from **removing waiting**, not from a faster robot.

How the 12 s became 8 s

- Shorter, blended moves (5.5): -1.4 s.
- Gripper valve closer to the tool: -0.8 s.
- Next part waits ready in a buffer: -1.8 s.

Parallel work: while the CNC machines part n, the robot checks and stacks part $n-1$.

Takt time and throughput: is the cell fast enough?



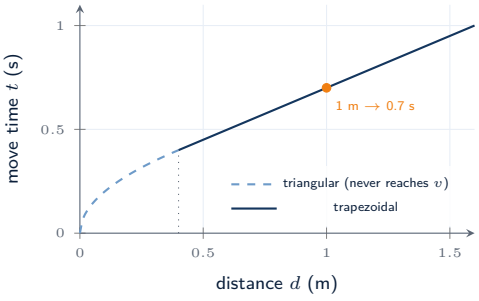
Takt time $t_{\text{takt}} = \dfrac{\text{available time}}{\text{demand}}$; need $t_c \le t_{\text{takt}}$

Worked example

1. Shift: 7.5 h = 27 000 s; demand 2700 parts/shift.
2. $t_{\text{takt}} = 27000/2700 = 10$ s (360 parts/h).
3. Cell cycle $t_c = 12$ s → $3600/12 = 300$ parts/h. **Too slow!**
4. Fix: shorten the cycle to ≤ 10 s (previous slide) or add a second robot.

Takt (German: beat) = the rhythm the customer demands.

Estimating a robot move time



Remember the trapezoidal velocity profile (4.7): accelerate, cruise, brake.

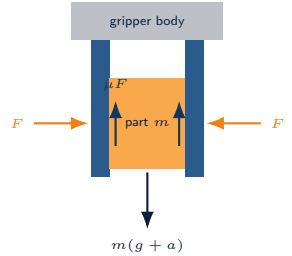
$$t = \frac{d}{v_{\max}} + \frac{v_{\max}}{a} \qquad \text{if } d \geq v_{\max}^2/a$$

Worked example

1. $v_{\max} = 2\,\text{m/s}$, $a = 10\,\text{m/s}^2$, $d = 1\,\text{m}$.
2. Check: $v^2/a = 0.4\,\text{m} \leq 1\,\text{m} \rightarrow$ trapezoid.
3. $t = 1/2 + 2/10 = \mathbf{0.7\,s}$.

Short moves are dominated by acceleration: halving the distance does *not* halve the time.

End-of-arm tooling (EOAT) and grip force



$$n\,\mu\,F \;\geq\; m\,(g+a)\,S$$

n friction surfaces, μ friction coeff., F finger force (N), a upward accel., S safety factor.

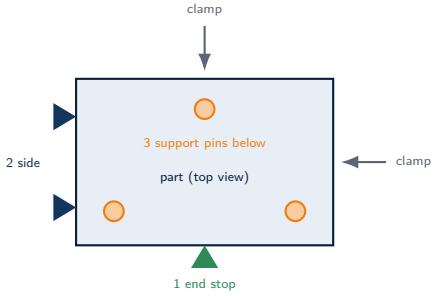
- EOAT: the robot's "hand" – fingers, vacuum cups, magnets, tools (W1D2).
- Usually custom-made for each part; often 10–30 % of cell cost.
- **Tool changer**: swap grippers automatically.

Worked example

1. $m = 2\,\text{kg}$, $a = 5\,\text{m/s}^2$, $\mu = 0.3$, $n = 2$, $S = 2$.
2. Needed friction: $2 \cdot (9.81 + 5) \cdot 2 = 59\,\text{N}$.
3. $F \geq 59/(2 \cdot 0.3) = \mathbf{99\,N} \approx 100\,\text{N}$ per finger.

Friction grip is weak: shaped fingers that "cradle" the part are much safer.

Fixtures and part presentation: the 3-2-1 rule



- $3 + 2 + 1$ points fix all 6 DOF of a rigid part (4.1).
- Then a clamp pushes the part against them.

- The robot repeats blindly: the part must be **in the same place every time**.
- **Fixture**: holds the part in a defined position.
- **Feeders**: bowl feeder, trays, conveyor with end stop.
- No fixture? Then a **vision** system must find the part (W2D5).

Take-away

Good fixtures make simple robot programs; bad fixtures cause most cell stops.

Check your understanding – 5.3

Check your understanding

1. A station is at $(1.2, 0.9)$ m from a robot with 1.6 m reach. Is it in the comfort zone?
2. Demand is 1800 parts in a 7.5 h shift. What is the takt time? Is a 14 s cycle enough?
3. Why does halving a short robot move not halve its time?

Discuss with your neighbour for 2 minutes.

Summary 5.3 – Work-cell design

Key take-aways

- A cell = robot + EOAT + fixtures + feeders + controls + guarding.
- Place stations at about 30–85 % of reach, with the right orientation.
- Cycle time must be $\leq$ takt time; remove waiting first.
- Move time $\approx d/v + v/a$ for long moves.
- Grip force from $n\mu F \geq m(g + a)S$; fixtures use the 3-2-1 rule.

Key terms

Work-cell – self-contained robot station.
EOAT – end-of-arm tooling.
Cycle time – time per finished part.
Takt time – time per part the demand allows.
Fixture – holds the part in place.
HMI – operator screen/panel.

Session 3

5.4 PLCs and Automation Control

Why PLCs? From relay cabinets to a rugged computer



Modular PLC (CPU + I/O modules). Palatinatian, CC BY 3.0, via Wikimedia Commons

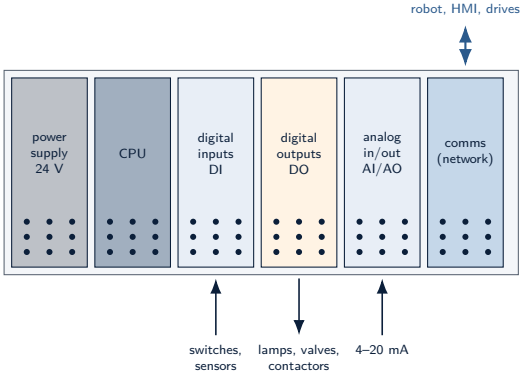
- Before 1970, machine logic was wired with hundreds of **relays**. A change = rewiring.
- 1968: General Motors asked for a programmable replacement → the **PLC**.

Definition

A **programmable logic controller** (PLC) is an industrial computer that reads sensors (inputs), runs a user program, and switches actuators (outputs) – reliably, in real time, for years.

- Survives heat, vibration, electrical noise.
- Programmed in languages engineers and electricians can read.
- In a robot cell: the PLC is the **conductor**, the robot is one of the musicians.

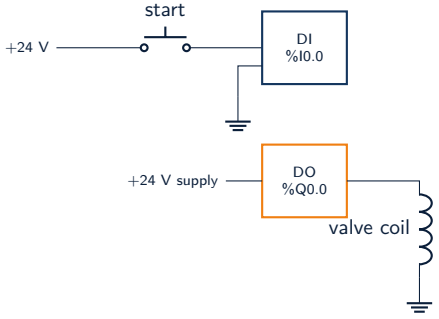
PLC hardware: a rack of modules



- **CPU**: runs the program, holds memory.
- **I/O modules**: the PLC's senses and muscles.
- Signals are usually **24 V DC**: 0 V = false, 24 V = true.
- **Fieldbus / network** (PROFINET, EtherNet/IP, EtherCAT) links robot, drives, HMI (6.7).

Going further: Small machines use compact PLCs (all in one box); large plants use remote I/O stations spread around the line.

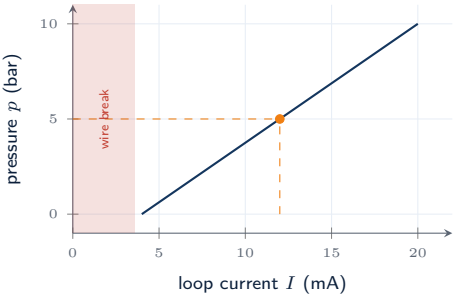
Digital inputs and outputs: wiring a sensor and a valve



- Input: button closed → 24 V at terminal → bit `%I0.0` = 1.
- Output: bit `%Q0.0` = 1 → module switches 24 V to the valve coil.
- Each I/O point has an **address** (vendor-specific form).
- **Sourcing** / **sinking**: whether the device supplies or receives the current – must match.

A digital signal is just one **bit**: ON/OFF, 1/0, TRUE/FALSE.

Analog signals: the 4–20 mA current loop



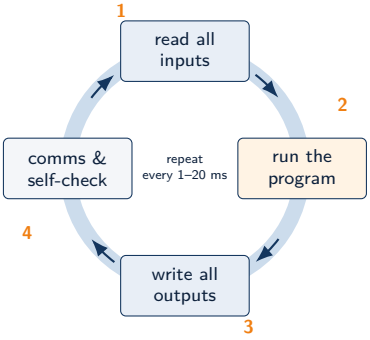
- A sensor sends a **current** proportional to the value: 4 mA = 0 %, 20 mA = 100 %.
- Current is not changed by long wires (unlike voltage).
- **Live zero**: 0 mA means a broken wire, not "zero pressure".

$$p = \frac{I - 4\,\text{mA}}{16\,\text{mA}} \cdot p_{\text{range}}$$

Example: range 0–10 bar, $I = 12$ mA:
$p = (8/16) \cdot 10 = \mathbf{5\ bar}$.

Going further: Inside, an ADC turns the current into an integer (W2D5); the program scales it to engineering units.

The scan cycle: how a PLC thinks



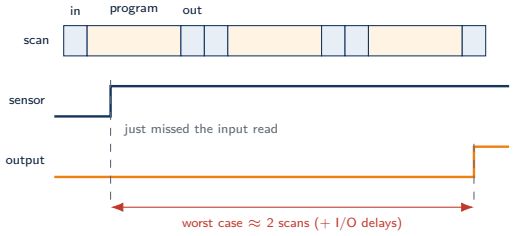
1. Copy all input terminals into memory (**process image**).
2. Execute the program top to bottom, using that snapshot.
3. Copy results to the output terminals.
4. Talk to the network, check itself; start again.

Analogy

A security guard on rounds: look at all cameras, decide, then act – and repeat. Anything that happens between two looks is seen on the next round.

Going further: A **watchdog** timer stops the PLC safely if one scan takes far too long.

Scan time and response time



$$t_{\text{resp,max}} \approx t_{\text{in}} + 2\,t_{\text{scan}} + t_{\text{out}}$$

Worked example

1. Input filter 3 ms, scan 10 ms, output 1 ms.
2. $t \approx 3 + 2 \cdot 10 + 1 = \mathbf{24\ ms}$.
3. A box on a conveyor at 0.5 m/s moves $0.5 \cdot 0.024 = 12$ mm before the stop gate reacts.

- A pulse *shorter* than one scan can be missed completely!
- Fast events → interrupt or high-speed counter inputs.

Refresher: Boolean logic

Refresher: three basic operations

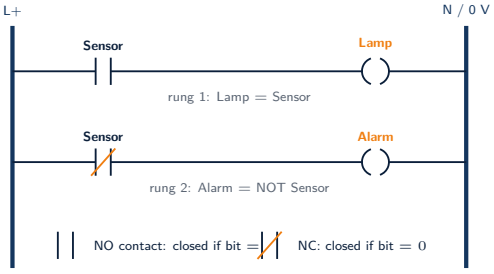
A	B	A AND B	A OR B	NOT A
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Written as $A \cdot B$ (AND), $A + B$ (OR), $\overline{A}$ (NOT).

- **AND**: all conditions true. "Door closed AND start pressed."
- **OR**: at least one true. "Button 1 OR button 2."
- **NOT**: inverts. "NOT emergency stop pressed."

Example: Run press = Start $\cdot$ DoorClosed $\cdot$ $\overline{\text{Fault}}$. Start $= 1$, DoorClosed $= 1$, Fault $= 0 \rightarrow 1 \cdot 1 \cdot 1 = 1$ $\rightarrow$ press runs.

Ladder logic: drawing a program like a relay circuit

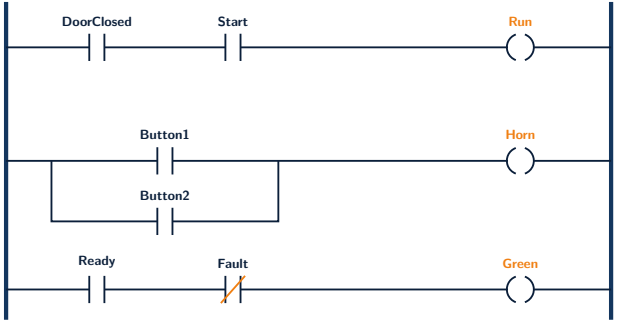


- Two vertical **rails** = "power"; horizontal **rungs** = logic lines.
- **Contacts** test a bit; a **coil** sets a bit.
- "Power" flows left to right if a path of closed contacts exists.
- Rungs are executed top to bottom, every scan.

Why ladder?

It looks like the relay drawings electricians already knew – and is still the most used PLC language.

AND, OR and NOT in ladder



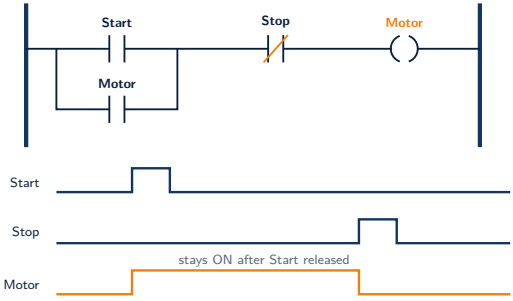
Run = DoorClosed AND Start

Horn = Button1 OR Button2

Green = Ready AND NOT Fault

- **Series** contacts = AND.
- **Parallel** branches = OR.
- **NC** contact = NOT.
- Any Boolean expression can be drawn this way.

The start/stop latch (seal-in): memory in one rung

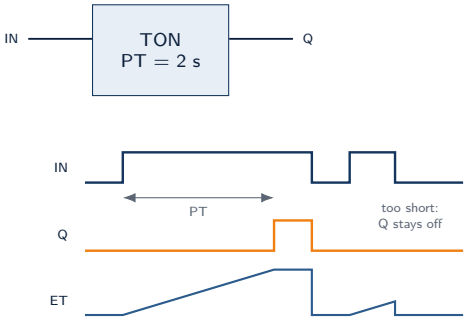


1. Press Start: Start contact closes → Motor coil ON.
2. The Motor contact (parallel to Start) now closes: **seal-in**.
3. Release Start: power still flows through the Motor contact.
4. Press Stop: NC contact opens → Motor OFF, seal broken.

> This is a 1-bit **memory**. Same idea: `SET`/`RESET` coils or an SR flip-flop.

Going further: A real stop button is wired NC (a broken wire then also stops the machine), so in the program it is tested with an NO contact. Emergency stops are never only in the PLC program – safety circuits, W2D4.

Timers and counters



- **TON** (on-delay timer): output Q turns on when IN has been true for the preset time PT. ET = elapsed time.
- Uses: "valve open for 2 s", "fault if part not seen within 5 s".
- **CTU** (up-counter): counts rising edges; Q = 1 when count $\geq$ preset.

Example – pallet full

Count each box placed (CTU, preset $= 4 \times 3 \times 5 = 60$). When Q $= 1$: stop the robot, request a new pallet, reset the counter.

IEC 61131-3: one standard, five languages

Language	Type	Good for
Ladder Diagram (LD)	graphical	simple on/off logic, interlocks; easy for electricians
Function Block Diagram (FBD)	graphical	signal flow, analog processing, reusable blocks
Structured Text (ST)	text (Pascal-like)	calculations, loops, data handling, state machines
Sequential Function Chart (SFC)	graphical	step-by-step sequences (next slides)
Instruction List (IL)	text (assembler-like)	legacy; deprecated since the 2013 edition

LD



FBD



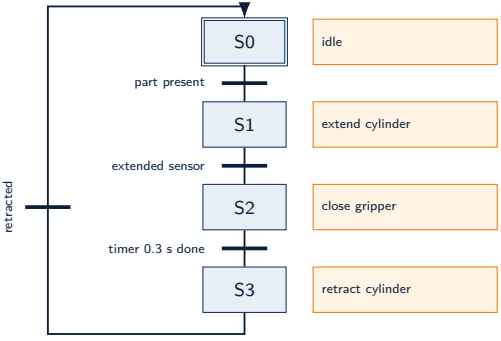
ST

```
Y := A AND B;
```

Same logic, three notations – the PLC does the same thing.

Going further: IEC 61131-3 also defines data types, function blocks (TON, CTU, . . .) and program organisation units, so code is portable between vendors – in principle.

Sequential control: steps and transitions



- Most machines work in **steps**: wait → extend → grip → retract → ...
- **Step**: what the machine is doing now (with actions).
- **Transition**: the condition that moves to the next step.
- Only one step active at a time (in this simple chain).

Analogy

A recipe: "boil water; *when it boils*, add pasta; *after 10 min*, drain."

This picture *is* an SFC program – and also a **state machine** (4 states).

Worked example: the same sequence in Structured Text

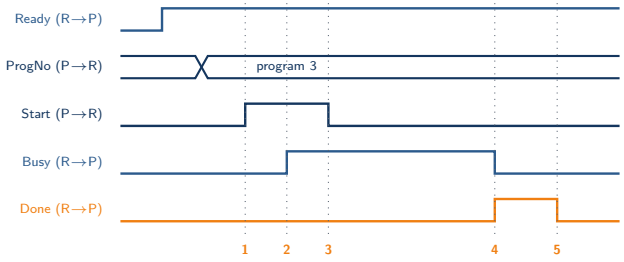
```
CASE step OF
  0:  (* S0 idle *)
    IF PartPresent THEN step := 1; END_IF
  1:  Extend := TRUE;
    IF IsExtended THEN step := 2; END_IF
  2:  Grip := TRUE;
    T1(IN := TRUE, PT := T#300ms);
    IF T1.Q THEN T1(IN := FALSE);
                 step := 3; END_IF
  3:  Extend := FALSE;
    IF IsRetracted THEN step := 0; END_IF
END_CASE
```

1. A variable `step` remembers which state we are in.
2. Each scan, only the code of the active step runs.
3. The `IF` line is the transition: if true, change `step`.
4. Outputs (`Extend`, `Grip`) are set in the step that needs them.

Trace: part arrives → step 1, cylinder extends (~0.4 s) → step 2, grip for 0.3 s → step 3, retract → back to 0. Total ~1.1 s.

Going further: Real programs add a fault step with a timeout in every state ("extended not seen within 2 s → alarm").

PLC ↔ robot: the handshake



1. PLC selects a program and sets **Start**.
2. Robot answers **Busy**.
3. PLC drops Start (it was seen).
4. Robot finishes: Busy off, **Done** on.
5. PLC reads Done, moves on to the next step.

Rule

Every command is **acknowledged**. Never assume the other side saw a signal – wait for its answer.

Check your understanding – 5.4

Check your understanding

1. A PLC scans every 20 ms. A sensor pulse lasts 5 ms. What can go wrong, and what is the fix?
2. Draw a rung: the Pump runs when (Level_Low OR Manual) AND NOT Overload.
3. A 4–20 mA sensor measures 0–100 °C. The PLC reads 16 mA. What is the temperature?

Discuss with your neighbour for 3 minutes.

Summary 5.4 – PLCs and automation control

Key take-aways

- A PLC reads inputs, runs logic and writes outputs in a repeating scan.
- Worst-case response $\approx$ 2 scans + I/O delays; short pulses can be missed.
- Ladder: series = AND, parallel = OR, NC = NOT; seal-in = memory.
- IEC 61131-3: LD, FBD, ST, SFC (IL deprecated).
- Sequences = steps + transitions; PLC and robot talk by handshake.

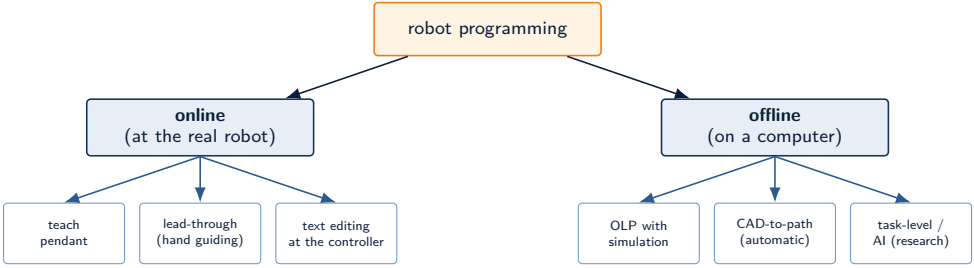
Key terms

PLC – programmable logic controller.
Scan cycle – read, execute, write, repeat.
Rung / contact / coil – ladder elements.
Seal-in – latch that holds itself on.
TON / CTU – on-delay timer / up-counter.
SFC – sequential function chart.

Session 4

5.5 Robot Programming

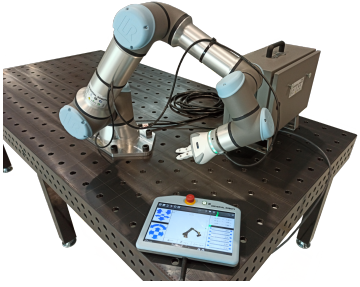
How do we tell a robot what to do?



- **Online**: simple, uses the real cell – but the cell stops while you program.
- Most installed robots are still taught online.

- **Offline**: program while production runs; needs an accurate model of the cell.
- Vendor languages: RAPID (ABB), KRL (KUKA), TP/KAREL (FANUC), URScript (UR).

The teach pendant



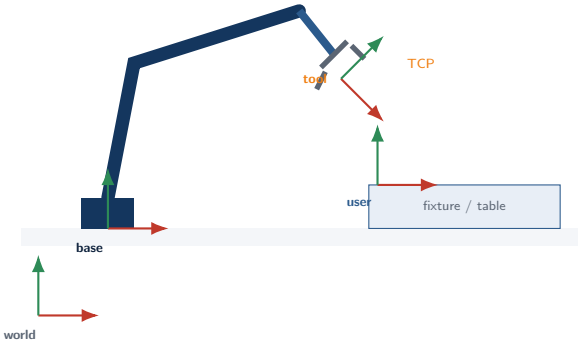
Cobot arm with its teach pendant (front). Auledas, CC BY-SA 4.0, via Wikimedia Commons

- Hand-held terminal: **jog** the robot, store points, edit and run programs.
- **Jog modes**: joint by joint, or the tool along x, y, z of a chosen frame.
- **Enabling switch** (3 positions): robot moves only while held in the middle – release *or* squeeze hard → stop.
- Manual mode **T1**: TCP speed limited to **250 mm/s** (ISO 10218-1).
- Emergency-stop button on every pendant.

Teach workflow

Jog to the point → check the tool angle → **store** the point → insert the move instruction → test slowly, step by step.

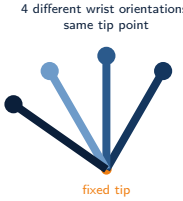
Coordinate frames: base, world, tool and user



- **World**: the cell's fixed reference (useful with several robots).
- **Base**: at the robot foot; the kinematics live here (4.3).
- **Tool** (TCP frame): at the working point of the gripper or torch.
- **User** / work-object: on the fixture or pallet.

Every taught point is a pose: **tool frame relative to a user frame**. The controller chains the homogeneous transforms (4.2): $T_{\text{base}}^{\text{tool}} = T_{\text{base}}^{\text{user}}\, T_{\text{user}}^{\text{tool}}$.

The tool frame (TCP) and why it must be right



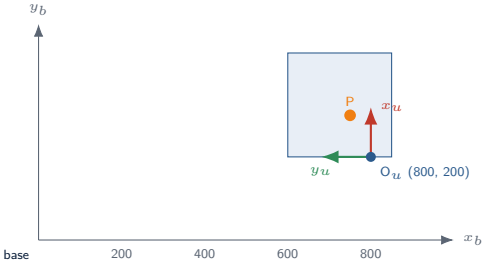
- **4-point method**: touch one fixed tip from 4 directions; the controller solves for the TCP offset.

- TCP = the point that moves along the programmed path and about which the tool rotates.
- Wrong TCP → the tool "wobbles" when it re-orients.

Worked example – TCP error

1. TCP length error $e = 5\,\text{mm}$.
2. Re-orient the tool by $\theta = 30^\circ$ "about the tip".
3. Real tip moves on an arc: chord $= 2e\sin(\theta/2)$.
4. $= 2 \cdot 5 \cdot \sin 15^\circ =$ **2.6 mm** – a missed weld seam!

User frames: teach once, move the fixture freely



- Teach 3 points: origin, a point on x_u, a point in the $x_u y_u$ plane.

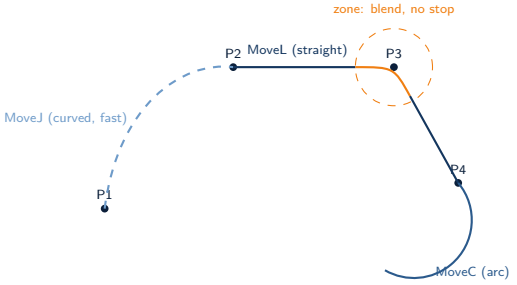
- Points are stored **relative to the fixture**.
- Fixture moved? Re-teach only its 3 frame points – all 50 program points follow.

Worked example

User frame: origin $(800, 200)$ mm, rotated $90°$ about z. Point P in user frame: $(100, 50)$ mm.

1. Rotate: $R(90°)\,(100, 50) = (-50, 100)$.
2. Add origin: $(800 - 50,\ 200 + 100)$.
3. P in base frame = **(750, 300)** mm.

Motion instructions: joint, linear, circular – and blending



- `MoveJ`/PTP: joints interpolated; fastest, path not straight – for free air moves.
- `MoveL`: TCP on a straight line – approach, welding, insertion.
- `MoveC`: arc through an auxiliary point.
- **Zone** / blending: pass *near* a via-point without stopping → shorter cycle.

Watch out

MoveL can drive the arm through a **singularity** (4.5); MoveJ cannot, but may swing wide.

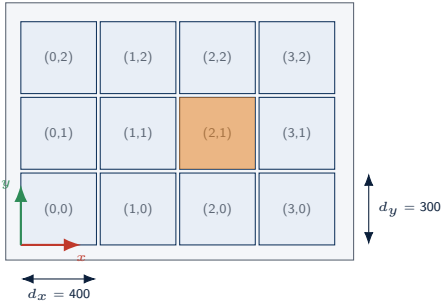
A complete pick-and-place program (generic pseudo-code)

```
MoveJ Home, v1000, fine;
LOOP
  WaitDI PartReady = 1;
  MoveJ Offs(Pick,0,0,100), v1000, z50;
  MoveL Pick, v200, fine;
  SetDO Gripper = 1; Wait 0.2;
  MoveL Offs(Pick,0,0,100), v500, z20;
  MoveJ Offs(Place,0,0,100), v1000, z50;
  MoveL Place, v200, fine;
  SetDO Gripper = 0; Wait 0.2;
  MoveL Offs(Place,0,0,100), v500, z20;
  PulseDO Done;
ENDLOOP
```

- `Offs(P,0,0,100)`: 100 mm above P – the **approach** point.
- `v200`: speed in mm/s; `fine`: stop exactly; `z50`: blend with 50 mm zone.
- `WaitDI`/`SetDO`: digital I/O – the link to the PLC (5.4).
- Only **two points** (Pick, Place) need teaching; the rest is calculated.

> Pattern: approach fast in joint mode, last centimetres slowly and straight, act, retreat straight.

Variables and loops: one program, a whole pallet



pallet user frame; boxes 400 × 300 × 200 mm

$$P_{ijk} = P_0 + (i\,d_x,\ j\,d_y,\ k\,d_z)$$

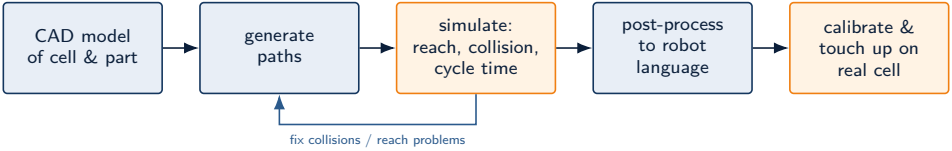
i = column, j = row, k = layer (counting from 0); P_0 = first box position.

Worked example

1. $P_0 = (200, 150, 200)$ mm (centre of first box top).
2. Box $i = 2$, $j = 1$, layer $k = 2$; $d_z = 200$ mm.
3. Offset $= (800, 300, 400)$ mm.
4. $P =$ **(1000, 450, 600)** mm in the pallet frame.

Three nested `FOR` loops place all $4 \times 3 \times 5 = 60$ boxes.

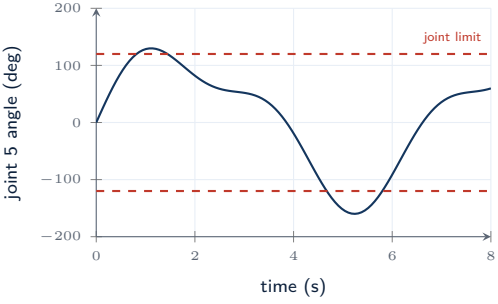
Offline programming (OLP): the workflow



Advantages	Challenges
No production stop while programming	Needs accurate models of robot, tools and cell
Test ideas and layouts before buying	Robot *accuracy* (not repeatability) now matters
Complex paths from CAD (welding, deburring)	Cables, hoses and real part variation are not in CAD

Going further: Robot calibration identifies the real link lengths and joint offsets, cutting absolute error from about 1 mm to a few tenths of a mm.

Simulation: checking reach, collisions and cycle time

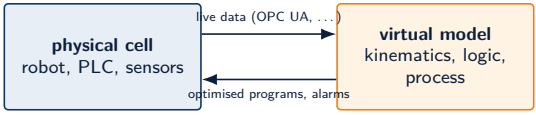


- **Reach & orientation**: can the tool reach every point at the right angle?
- **Joint limits & singularities**: the plot shows joint 5 exceeding its limit → change the path or the part orientation.
- **Collisions**: robot, tool, fence, machine.
- **Cycle time**: realistic within a few % if the controller model is good.

Analogy

A flight simulator for the robot: crash it on the computer, not in the cell.

From simulation to digital twin



model $\longrightarrow$ (one-way) simulation;
continuously synchronised $\longrightarrow$ **digital twin**

- **Virtual commissioning**: test the real PLC program against a simulated cell before installation.
- **Digital twin**: model kept in sync with the real cell by live data.
- Uses: predict cycle time, spot wear, try changes safely (Industry 4.0, 5.1).

> Virtual commissioning can move much of the on-site debugging to the office – the real cell starts producing sooner.

Check your understanding – 5.5

Check your understanding

1. A fixture is moved 50 mm. Why does a program written in a user frame need only 3 points re-taught?
2. Which move type would you use to insert a pin into a hole – and which to swing back to Home? Why?
3. Give one reason why an offline program may miss the part on the real robot.

Discuss with your neighbour for 3 minutes.

Summary 5.5 – Robot programming

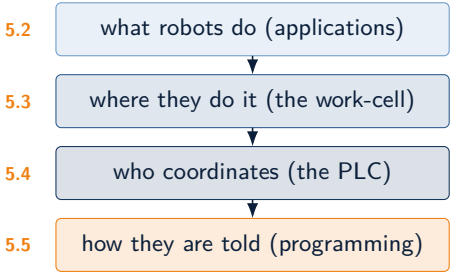
Key take-aways

- Online (teach pendant, lead-through) vs. offline (OLP + simulation).
- Frames: world, base, tool (TCP), user; points are stored relative to them.
- Correct TCP and user frames make programs robust and portable.
- MoveJ for free moves, MoveL/MoveC for process paths; zones save time.
- Simulation checks reach, collisions, cycle time; a digital twin stays in sync.

Key terms

Teach pendant – hand-held programming unit.
Jog – move the robot manually.
TCP – tool centre point.
User frame – frame on the work-piece.
Zone – blend radius at a via-point.
OLP – offline programming.

Day summary



Five things to remember

1. The process decides the robot (payload, path, speed).
2. Cycle time $\leq$ takt time; check reach in the layout.
3. PLC: read – execute – write, every few ms.
4. Ladder: series AND, parallel OR, seal-in memory.
5. Teach points relative to good tool and user frames.

Preview of Week 2 Day 4

Tomorrow: manufacturing (cont.) and sensors I

- 5.6 Material handling: conveyors, AGV/AMR, FMS cells, cobots and human–robot collaboration.
- 5.7 Safety and standards: risk assessment, ISO 10218, ISO/TS 15066, performance levels, light curtains, safety PLC.
- 6.1–6.3 Sensors: accuracy and resolution; encoders, IMUs; force, proximity and range sensors.

Think about it

Today's cell had a fence. What would have to change for a person to work *next to* the robot without one?

Figure credits

All plots, layouts, ladder diagrams, timing diagrams and schematics without a credit line are original drawings made for this course. Downloaded figures (Wikimedia Commons):

File	Source (Wikimedia Commons) and author	Licence
r23_spot_welding.jpg	File:BMW Leipzig MEDIA 050719 Download Karosseriebau max.jpg – BMW Werk Leipzig	CC BY-SA 2.0 de
r23_painting.jpg	File:Japan- Painting Robot.jpg – Gary Todd	CC0
r23_palletising.jpg	File:Factory Automation Robotics Palettizing Bread.jpg – KUKA Roboter GmbH, Bachmann	Public domain
r23_delta.jpg	File:Delta x robot - vision module - conveyor.jpg – Trungdoanhong	CC BY-SA 4.0
r23_plc.jpg	File:S71500.JPG – Palatinatian	CC BY 3.0
r23_pendant.png	File:UR16e robot arm.png – Auledas	CC BY-SA 4.0

References

- Groover, M. P.: *Automation, Production Systems, and Computer-Integrated Manufacturing*, 4th ed. Pearson, 2015.
- Nof, S. Y. (ed.): *Handbook of Industrial Robotics*, 2nd ed. Wiley, 1999.
- Siciliano, B.; Khatib, O. (eds.): *Springer Handbook of Robotics*, 2nd ed. Springer, 2016.
- Craig, J. J.: *Introduction to Robotics: Mechanics and Control*, 4th ed. Pearson, 2018.
- Boothroyd, G.: *Assembly Automation and Product Design*, 2nd ed. CRC Press, 2005.
- Whitney, D. E.: *Mechanical Assemblies*. Oxford Univ. Press, 2004.
- International Federation of Robotics: *World Robotics – Industrial Robots* (annual report).
- Pan, Z. et al.: Recent progress on programming methods for industrial robots. *Robotics and Computer-Integrated Manufacturing* 28(2), 2012.
- Bolton, W.: *Programmable Logic Controllers*, 6th ed. Newnes, 2015.
- Petruzella, F. D.: *Programmable Logic Controllers*, 5th ed. McGraw-Hill, 2016.
- John, K.-H.; Tiegelkamp, M.: *IEC 61131-3: Programming Industrial Automation Systems*, 2nd ed. Springer, 2010.
- IEC 61131-3:2013 Programmable controllers – Part 3: Programming languages.
- ISO 9283:1998 Manipulating industrial robots – Performance criteria and related test methods.
- ISO 9787:2013 Robots and robotic devices – Coordinate systems and motion nomenclatures.
- ISO 10218-1:2011 Robots and robotic devices – Safety requirements for industrial robots – Part 1: Robots.
- Grieves, M.; Vickers, J.: Digital twin: mitigating unpredictable, undesirable emergent behavior in complex systems. In *Transdisciplinary Perspectives on Complex Systems*, Springer, 2017.